COMP4442 Service and Cloud Computing
Lab 6

Weiyang MA

April 8, 2026

Questions in Lab5

- What is Narrow Dependencies and Wide Dependencies of RDDs? Which RDDs operations belongs to Narrow Dependencies and Wide Dependencies, respectively?
  - **Narrow dependencies**: Each partition of the parent RDD is used by at most one child partition; typical operations include map, filter, and union.
  - **Wide dependencies**: Multiple child partitions depend on the same parent partition, requiring data shuffling; typical operations include groupByKey, reduceByKey, and join.
- What is lazy evaluation of RDDs? What is the benefits of lazy evaluation?
  - **Lazy evaluation**: Transformations on RDDs are not executed immediately but are deferred until an action is called.
  - **Benefits**: It enables optimization of execution plans and reduces unnecessary computation, improving overall efficiency.

Outline

- Review of Lab5

- How to develop web app to visualize real-time data
  - Store data constantly to the database
  - Build a HTML chart with JavaScript
  - Create web app to visual real-time data with flask
  - Deploy code in AWS Beanstalk

Review of Lab5

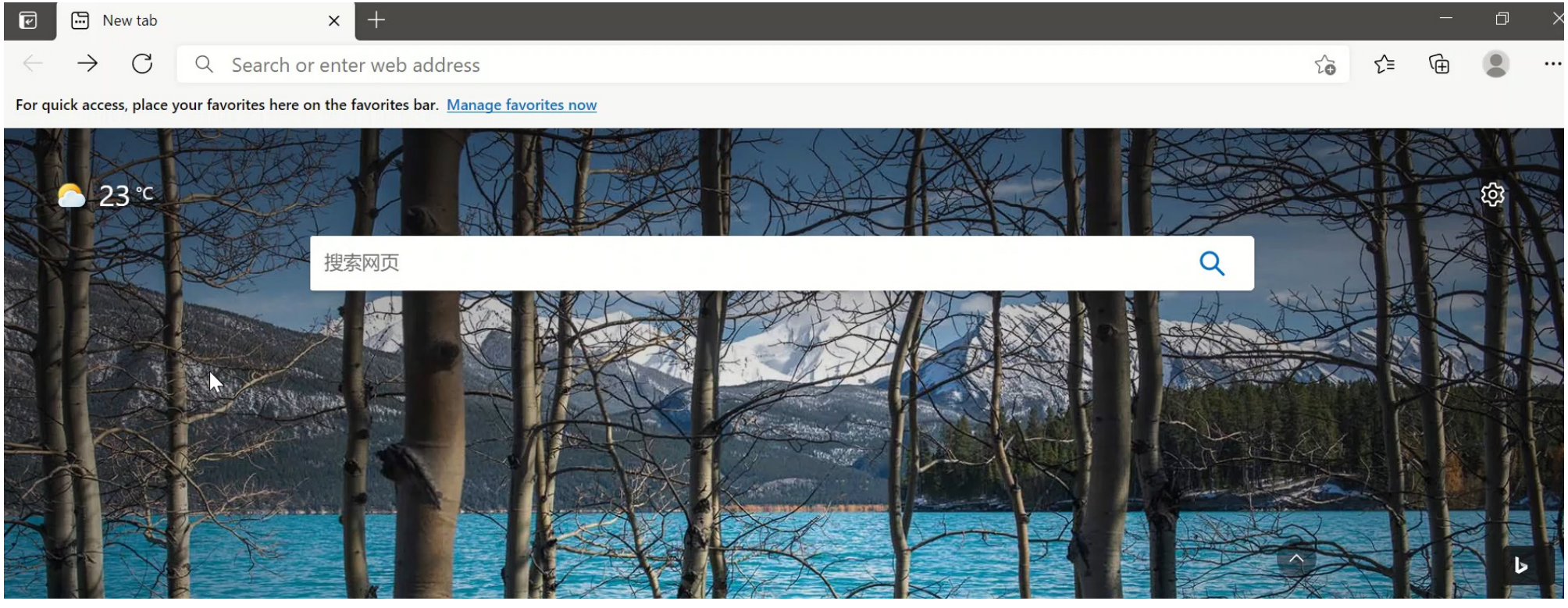
- We learned how to use AWS Spark to count the frequency of words in large file.
- But how to visual the analytical results, e.g., in a webpage

```
import os
import sys

from pyspark import SparkContext

args = sys.argv
inp = args[1]
out = args[2]

sc = SparkContext()

text_file = sc.textFile(inp)
counts = text_file.flatMap(lambda line: line.split(" ")) \
             .map(lambda word: (word, 1)) \
             .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile(out)

sc.stop()
```

```
('Gutenberg', 7)
('Complete', 3)
('Works', 3)
('William', 64)
('Shakespeare,', 1)
('by', 2791)
('This', 1101)
('eBook', 2)
('for', 5587)
('the', 23104)
('cost', 32)
('and', 18173)
('with', 6701)
('almost', 135)
('You', 1508)
('copy', 14)
('it', 4875)
('or', 1717)
('under', 196)
('terms', 45)
('License', 1)
('included', 1)
('**', 4)
('a', 12472)
```

Visualize Real-time Data with Flask

- We will develop a web app to visualize real-time data
  - A database to store the constantly generated real-time data
  - A HTML chart to visualize and monitor the real-time data



Lab Preparation

- Configure the develop environment locally
  - Create an empty directory *lab6* and go to the directory `cd lab6`
  - Install the virtual environment `pip install virtualenv`
  - Create a virtual environment `virtualenv lab6`
  - Activate the virtual environment
    - `lab6\Scripts\activate` (windows)
    - `Source lab6/bin/activate` (macOS)
  - Install flask `pip install flask`
  - Install mysql-connector `pip install mysql-connector-python`

Store Data Constantly to Database

- Create a new table in a new database
  - By Python or Software

```
drop database if exists lab6;
create database if not exists lab6;
use lab6;
CREATE TABLE IF NOT EXISTS Monitor
(
id int(11) unsigned NOT NULL AUTO_INCREMENT,
num int(11) DEFAULT NULL,
ctime bigint(11) DEFAULT NULL,
PRIMARY KEY (id)
) ENGINE=InnoDB AUTO_INCREMENT=0 DEFAULT CHARSET=utf8;
```

Store Data Constantly to Database

- Create the *write.py* file to insert data to the database

```python
import time
import mysql.connector
import json
import random

def db_connection():
    mydb = mysql.connector.connect( host = 'comp4442-lab3.cicfnyxayefu.us-east-1.rds.amazonaws.com',
    user = 'admin',
    port = '3306',
    database = 'lab6',
    passwd = '12345678',
    autocommit = True)
    return mydb
mydb = db_connection()
cur = mydb.cursor()

def genData():
    number=15 + 5*random.randint(1,10)
    Time = int(time.time())
    data = {}
    data['time'] = Time
    data['number'] = number
    return data

def execute():
    data = genData()
    sql = "insert into Monitor(num,ctime) values ({0},{1})".format(data['number'],data['time'])
    ret = cur.execute(sql)

while True:
    execute()
    time.sleep(1)
```

→ Database connection

→ Generate real-time data

→ Insert data into database

Build a HTML chart with JavaScript

- Then, we create the HTML chart (Highchart)

```html
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Real-time Monitoring</title>
  <script src="http://cdn.hcharts.cn/jquery/jquery-1.8.3.min.js"></script>
  <script src="http://cdn.hcharts.cn/highstock/highstock.js"></script>
  <script src="http://cdn.hcharts.cn/highcharts/modules/exporting.js"></script>
</head>

<body>
<div id="container" style="min-width:400px;height:400px"></div>
</body>

<script type="text/javascript">
$(function () {
  $.getJSON('/data', function (data) {
    // Create the chart
    $('#container').highcharts('StockChart', {
      chart: {
        events: {
          load: function () {
            var chart = $('#container').highcharts();
            var series = chart.series[0];
            //2 seconds interval
            setInterval(function () {
              $.getJSON("/data", function (res) {
                $.each(res, function (i, v) {
                  series.addPoint(v);
                });
              });
            }, 2000);
          }
        }
      },
      rangeSelector : {
        selected : 1
      },
      title : {
        text : 'Real-time Monitoring'
      },
      series : [{
        name : 'Real-time Number',
        data : data,
        tooltip: {
          valueDecimals: 2
        }
      }]
    });
  });
});
</script>
</html>
```

Claim the location of Highchart JavaScript

Set the interval as 2s, asynchronous update

Visual Real-time Data with Flask

- Bind the function `getdata()` to the url "`/data`"

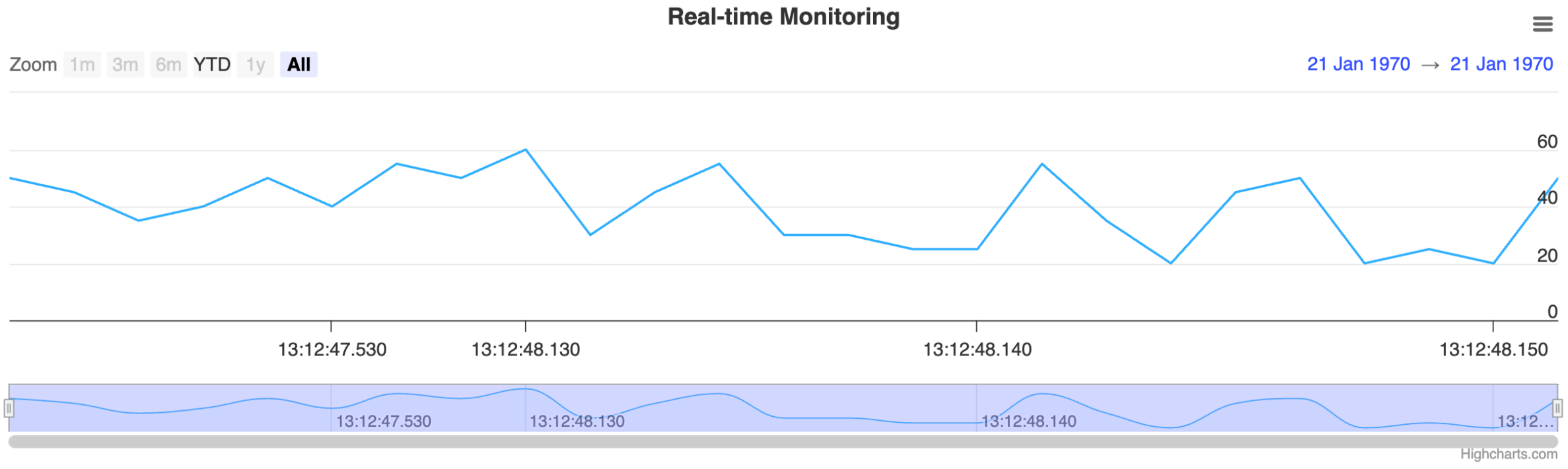
```python
from flask import Flask, request, render_template
import json
import mysql.connector

application = Flask(__name__)

def db_connection():
    mydb = mysql.connector.connect( host = 'comp4442-lab3.cicfnyxayefu.us-east-1.rds.amazonaws.com',
    user = 'admin',
    port = '3306',
    database = 'lab6',
    passwd = '12345678',
    autocommit = True)

    #print("successfully connect to the database")

    return mydb

mydb = db_connection()
cur = mydb.cursor()

@application.route("/")
def index():
 return render_template("monitor.html")

tmp_time = 0

@application.route("/data")
def getdata():
    global tmp_time
    if tmp_time > 0 :
        sql = "select ctime,num from Monitor where ctime >%s" %(tmp_time)
    else:
        sql = "select ctime,num from Monitor"

    cur.execute(sql)
    datas = []
    for i in cur.fetchall():
        datas.append([i[0], i[1]])

    if len(datas) > 0 :
        tmp_time = datas[-1][0]

    return json.dumps(datas)


if __name__ == "__main__":
    application.run(port=5000,debug=True)
```

→ Database connection

→ Query the data from the database and update the front-end webpage

Visual Real-time Data with Flask

- Check the chart to visualize data at "*http://127.0.0.1:5000/*".



Visual Real-time Data with Flask

- Check the data at "*http://127.0.0.1:5000/data*".

[[1775569522, 20], [1775569523, 55], [1775569524, 35], [1775569526, 30], [1775569527, 35], [1775569528, 30], [1775569529, 50], [1775569530, 50], [1775569532, 55], [1775569533, 55], [1775569534, 25], [1775569535, 40], [1775569537, 55], [1775569538, 65], [1775569539, 55], [1775569540, 65], [1775569542, 50], [1775569543, 55], [1775569544, 45], [1775569545, 65], [1775569546, 65], [1775569548, 30], [1775569549, 35], [1775569550, 60], [1775569551, 40], [1775569552, 20], [1775569554, 20], [1775569555, 45], [1775569556, 45], [1775569557, 30], [1775569559, 35], [1775569560, 65], [1775569561, 60], [1775569562, 55], [1775569563, 25], [1775569565, 50], [1775569566, 50], [1775569567, 20], [1775569568, 60], [1775569570, 35], [1775569571, 50], [1775569572, 55], [1775569573, 50], [1775569575, 30], [1775569576, 50], [1775569577, 60], [1775569578, 25], [1775569579, 40], [1775569581, 65], [1775569582, 55], [1775569583, 40], [1775569584, 65], [1775569586, 25]]

Deploy Code in AWS Beanstalk

- To deploy the code, we need to configure the requirements.txt.
  - `pip freeze > requirement.txt`
- Pack the following files and directories into a ZIP file.

requirement.txt

```
blinker==1.9.0
click==8.3.2
Flask==3.1.3
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.3
mysql-connector-python==9.6.0
Werkzeug==3.1.8
```

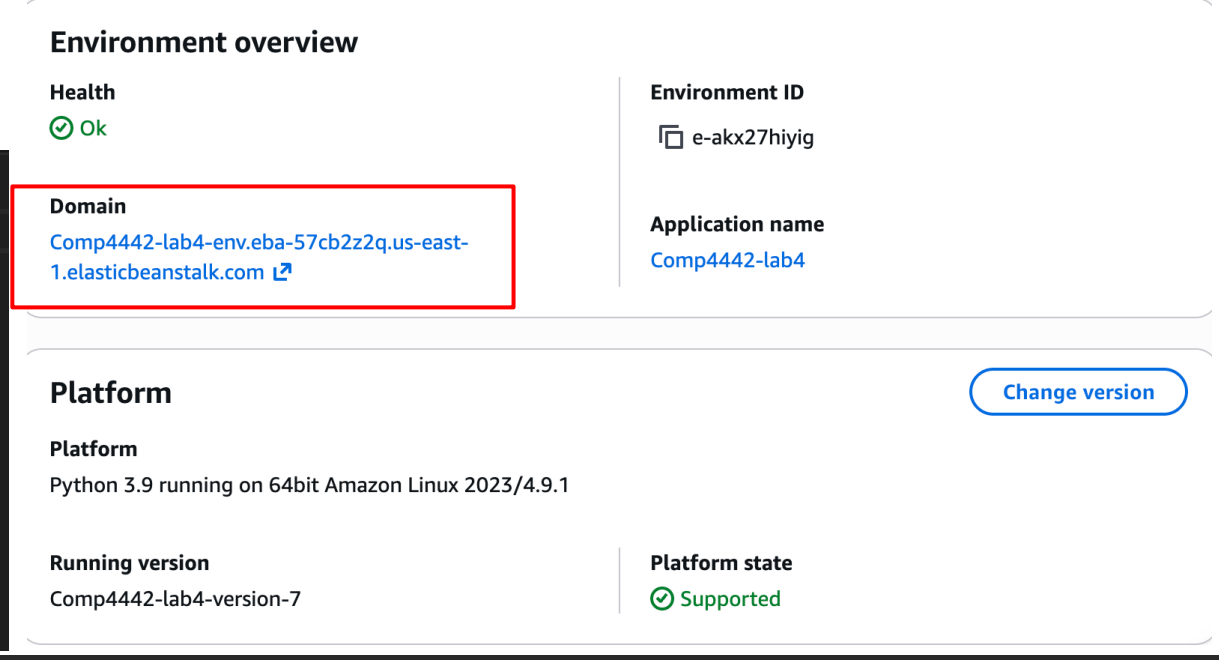
Name	Date modified	Type	Size
lab6	4/13/2021 8:06 PM	File folder	
templates	4/13/2021 4:42 PM	File folder	
application.py	4/13/2021 9:06 PM	Python File	2 KB
lab6_project.zip	4/13/2021 9:09 PM	Compressed (zipped)...	3 KB
requirements.txt	4/2/2021 10:05 PM	Text Document	1 KB
test.py	4/13/2021 9:04 PM	Python File	1 KB
write.py	4/13/2021 10:09 PM	Python File	1 KB

Deploy Code in AWS Beanstalk

- Restore Beanstalk environment and upload the ZIP file as we do in lab-01 and lab-04 or Create a new environment.
- Run the write.py locally.
- Open the website and observe.
- Running on Python3.9 successfully

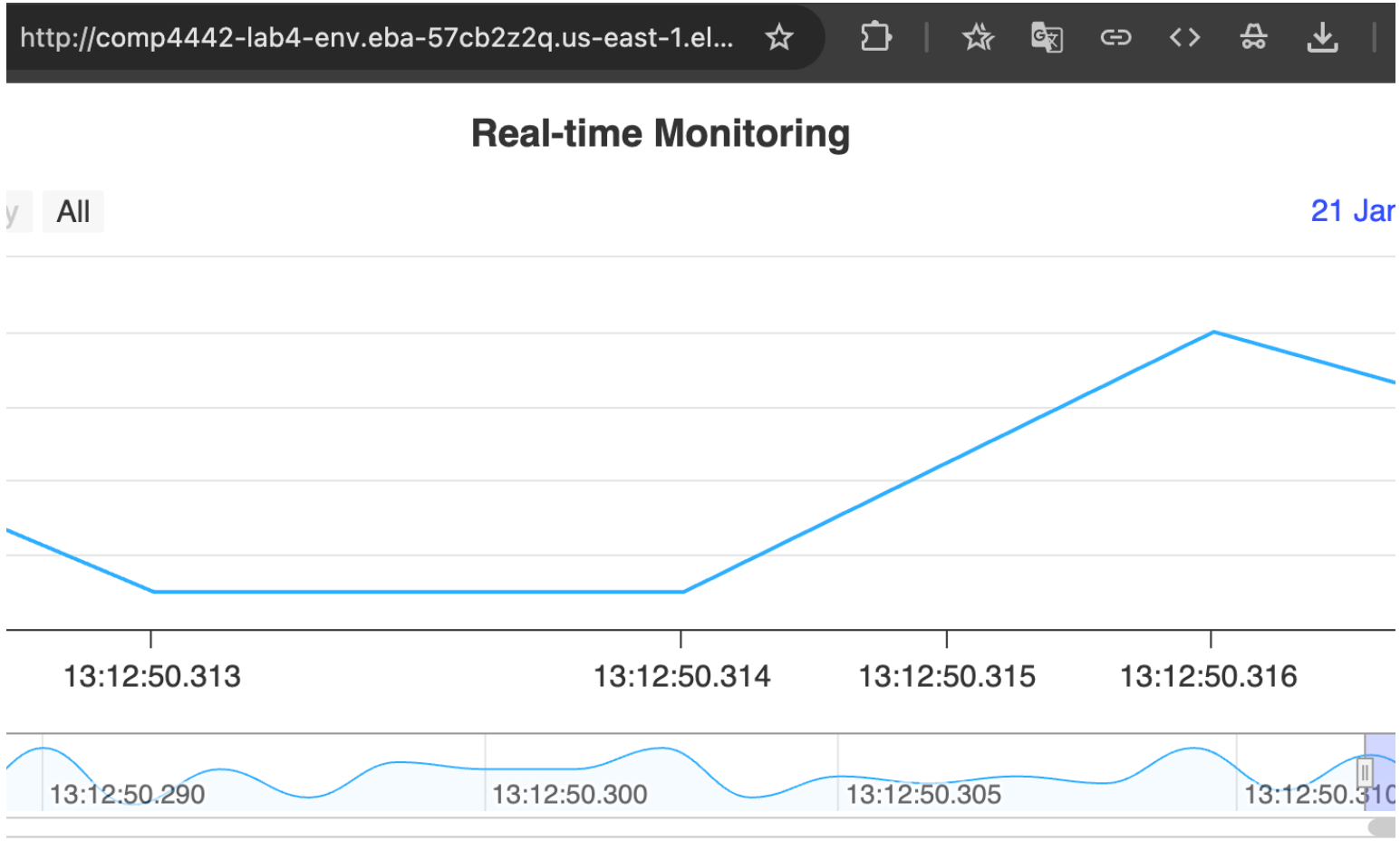
requirement.txt

```
blinker==1.9.0
click==8.3.2
Flask==3.1.3
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.3
mysql-connector-python==9.6.0
Werkzeug==3.1.8
```



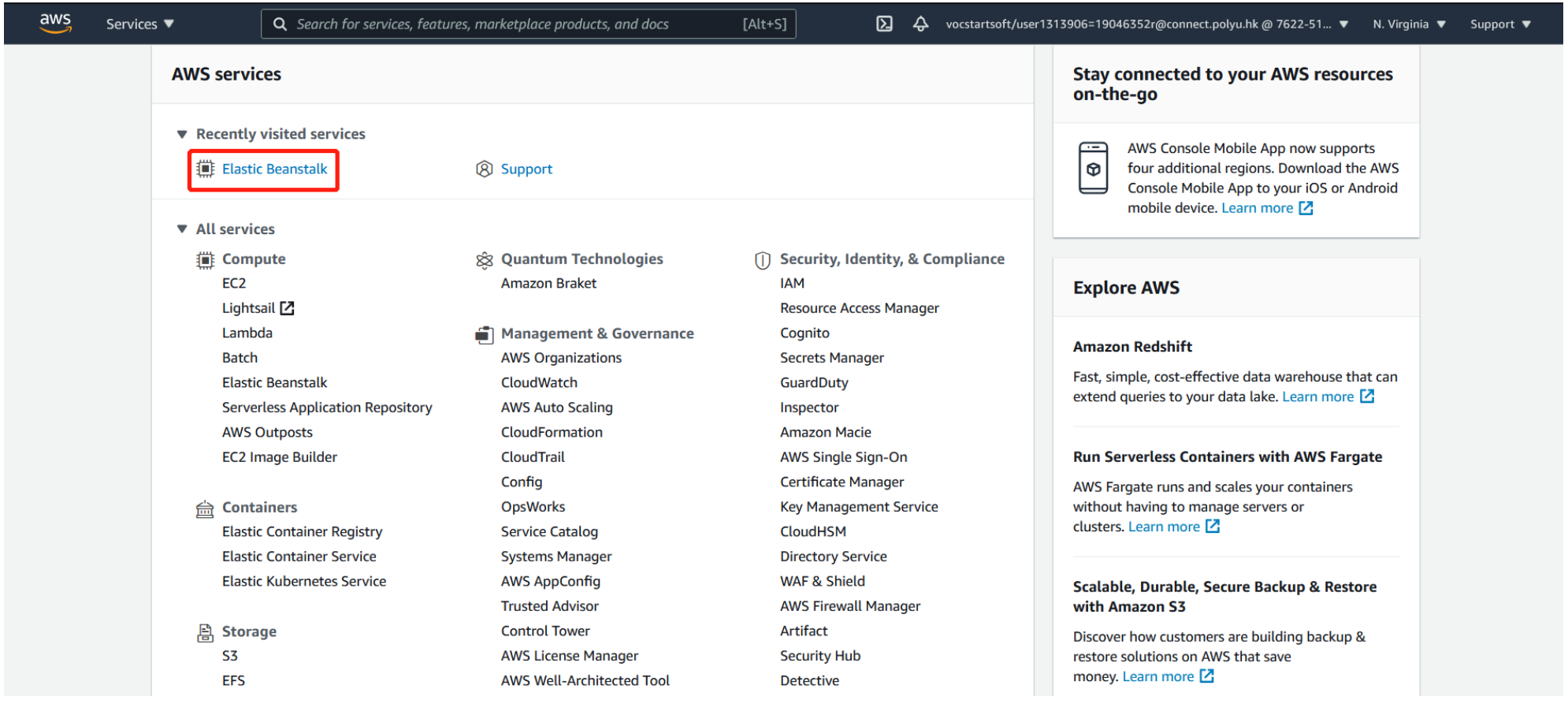
Deploy Code in AWS Beanstalk

- Web app to visualize real-time data.



Stop Elastic Beanstalk

- Access the beanstalk



Stop Elastic Beanstalk

- Terminate the environment

aws Services Search for services, features, marketplace products, and docs [Alt+S] vocstartsoft/user1313906=19046352r@connect.polyu.hk @ 7622-51... N. Virginia Support

Elastic Beanstalk
Environments
Applications
Change history
Recent environments
Comp4442lab1-env

Elastic Beanstalk > Environments

All environments

2 Actions Create a new environment

Load configuration
Save configuration
Swap environment URLs
Clone environment
Abort current operation
Restart app server(s)
Rebuild environment
3 Terminate environment

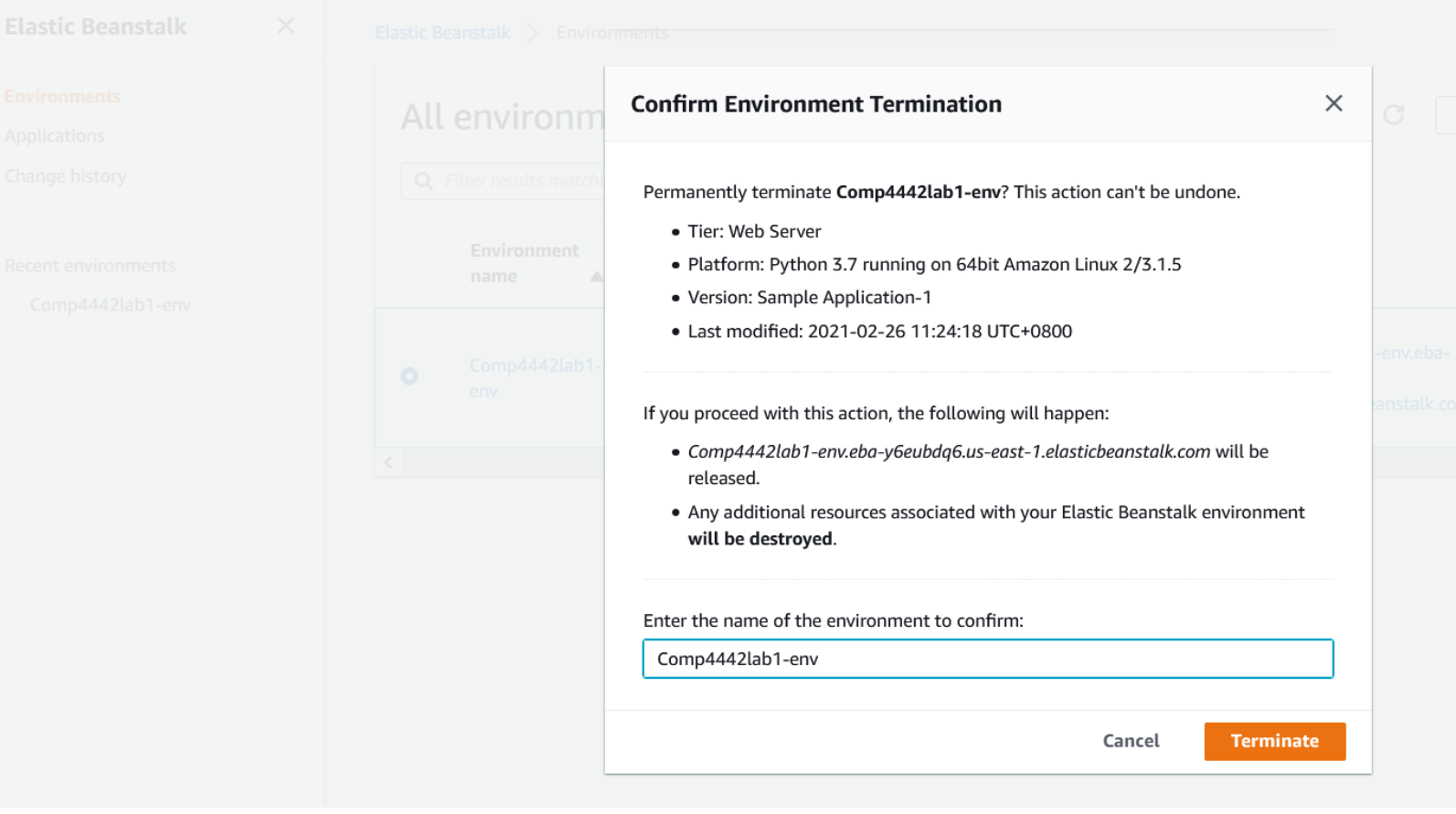
Filter results matching the display values

1

Environment name	Health	Application name	Date created	Last modified	URL	Platform	Platfo state
Comp4442lab1-env	Ok	COMP4442-lab1	2021-02-26 11:12:43 UTC+0800	2021-02-26 11:24:18 UTC+0800	Comp4442lab1-env y6eubdq6.us-east-1.elasticbeanst	Python 3.7 running on 64bit Amazon Linux 2	Suppo

Stop Elastic Beanstalk

- Confirm your operation



Lab Overview: What to Learn

- Introduction to Amazon Web Services (AWS)
- Data storage and management via AWS S3
  - Learn how to upload, access, and migrate data on AWS
- Create and manage databases via AWS RDS
  - Learn how to make use of database to support complex applications
- Building a dynamic web app via AWS Beanstalk
  - Learn how to use web server to deploy applications
- Big Data Analytics via AWS Spark
  - Learn how to use spark to analyze big data
- Visualize real-time data with dynamic web app

Q&A